On the Weak Diffusion of the SEPAR Cipher

Noah Ostle

Under supervision of Dr. Luke Mathieson, UTS School of Computer Science

Abstract

SEPAR encrypts each 16-bit plaintext word through eight keyed permutations, called rotors, each preceded by a state addition. Each rotor uses a 32-bit segment of the master key. We show that poor diffusion in the rotor's linear layer preserves classes of inputs throughout the rotor. Under suitable IVs, the ciphertext reveals the cyclic order of these classes. The main result is a search that turns this order into rotor key candidates. Applying it successively gives a chosen IV, chosen plaintext attack on the full 256-bit master key.

For the stated sampling budgets, the returned list contains the true key with probability at least $1 - 1.663852 \times 10^{-8} \approx 99.99999834\%$. In 500 trials, the C implementation returned a single key every time, using an average of $2^{27.83}$ queries and 81.48 seconds on one thread of an AMD Ryzen 9 8945HS processor. A general bound on the number of surviving candidates remains open.

1 Prior Work

1.1 Existing Security Analysis

SEPAR was proposed by Arsalan Vahi and Somaye Jafarali Jassbi in *Wireless Personal Communications* [1]. We examine the proposal's security claims and the later differential analysis by Vahi and Mirnia [2].

Vahi and Mirnia's *On the Differential Cryptanalysis of SEPAR Cipher* is a 2021 preprint extending the analysis in the published SEPAR proposal. Its aim is to justify resistance to differential cryptanalysis by counting active S-boxes. Table II of that study reports maximum differential counts $16370, 1016, 84, 22, 22$ for zero through four `b16` layers. Table III lists seven input and output difference pairs. In Section III-B, Vahi and Mirnia claim at least ten active S-boxes across four `b16` layers, then at least eighty across eight Enc-blocks. Using a maximum S-box transition probability of 2^{-2}, they derive

$$(2^{-2})^{80} = 2^{-160}.$$

They present this as evidence of resistance to differential attack.

The argument combines a bound on individual S-box transitions with a count of the S-boxes through which a difference must pass. The first comes from the substitution tables, and the second depends on the linear layer.

In their terminology, a `b16` is one internal substitution and mixing layer. An Enc-block is a complete rotor. Their active S-box argument means to extend the analysis of one Enc-block to the eight rotors used to encrypt each block.

1.2 Active S-boxes

For a function F on n-bit words, define its difference count by

$$D_F(\alpha, \beta) = \#\{x : F(x) \oplus F(x \oplus \alpha) = \beta\}.$$

For uniform x, the corresponding differential probability is $D_F(\alpha, \beta)/2^n$. An S-box is *active* when its input difference is nonzero. XOR with the subkey does not change that difference. Substitution changes its value, and the linear layer determines where it spreads.

For a 4-bit S-box, the stated maximum probability 2^{-2} corresponds to a difference count of 4 among 16 possible inputs. The linear layer determines how many of these substitutions a difference reaches in later layers.

A *differential characteristic*, also called a trail, specifies the differences at intermediate stages as well as at the endpoints. Its probability is commonly estimated by multiplying the S-box transition probabilities along it. Whether this product gives a bound depends on the dependencies between the transitions.

A differential fixes only the input and output differences. For a fixed key, its probability sums over all compatible intermediate trails. Thus a bound on individual trails must account for their combined probability before it can bound a differential. This distinction matters when interpreting Vahi and Mirnia's 2^{-160} estimate.

1.3 The diffusion claim

Section III-B of Vahi and Mirnia's paper starts from the seven difference pairs in their Table III, which they describe as results of an exhaustive search for high probability characteristics [2]. Table 1 gives these pairs in hexadecimal, with one digit for each nibble. The last column counts the nonzero input nibbles.

Pair	Input difference	Output difference	Nonzero input nibbles
1	`0424`	`2A5A`	3
2	`0494`	`2A5A`	3
3	`0704`	`5D93`	2
4	`0B24`	`2A5A`	3
5	`0B94`	`2A5A`	3
6	`0E04`	`5D93`	2
7	`CC80`	`61E6`	3

Table 1: The endpoint differences in Vahi and Mirnia's Table III for an Enc-block with four `b16` layers [2]. The nibble counts in the last column were calculated based on their results.

Their proof assigns contributions of three and four active S-boxes to the first and last layers, then considers totals $i = 3, 4, 5$ in the two middle layers. For $i = 3$, the proposed patterns are

$$3 + 2 + 1 + 4 = 10, \qquad 3 + 1 + 2 + 4 = 10.$$

For $i = 4$, they give $3 + 2 + 2 + 4$, $3 + 1 + 3 + 4$, and $3 + 3 + 1 + 4$, each totalling eleven. The smallest total is ten, which they multiply by eight to obtain eighty active S-boxes for the cipher [2].

The analysis does not establish why the first and last contributions must be three and four, or why the two middle layers must contribute at least three. To prove the minimum, the argument must rule out every pattern with fewer than ten active S-boxes. Listing several patterns whose totals are ten or more does not do this. The positions of the active nibbles matter as well as their number, since different positions can propagate differently through the same linear layer.

There is already a problem with inferring the first contribution from the reported pairs; inputs `0704` and `0E04` have only two nonzero nibbles. Each therefore activates two S-boxes in the first substitution layer. All seven output differences have four nonzero nibbles, but an output difference does not specify the inputs to the last `b16` layer.

More generally, the two endpoints of a differential do not determine its intermediate trail. Table III gives neither those intermediate differences nor a probability for each listed pair. The table alone therefore cannot establish the proposed activity patterns or show that the minimum is attained. Although Section III-A mentions mixed-integer linear programming in SageMath as a way to count active S-boxes, the SEPAR argument presented in Section III-B is the by-hand count above, and no model or solver result is supplied [2].

To add confidence to the notion that these estimations are not strong evidence for differential resistance, Section 3.2 gives a counterexample as the rotor in the authors' code admits a trail with only four active S-boxes across its four internal layers.

1.4 Interpreting the reported differential counts

Table II of the preprint reports the results reproduced in Table 2. Section III attributes this numerical study to the earlier SEPAR proposal. The authors interpret the unchanged maximum

at three and four `b16` layers as evidence that four layers provide sufficient resistance to differential cryptanalysis [2].

`b16` layers	Reported maximum count	Count divided by 2^{16}
0	16370	0.249786
1	1016	0.015503
2	84	0.001282
3	22	0.000336
4	22	0.000336

Table 2: Counts from Vahi and Mirnia's Table II [2]. The last column divides each count by 2^{16}, to six decimal places. These are differential probabilities if the counts use all input words.

The method used to obtain the reported maximum of 22 is unclear.

Under the definition of D_F above, an exhaustive calculation for one fixed key would first evaluate the Enc-block on all 65536 inputs. For each nonzero input difference α, it would then count the values of

$$F(x) \oplus F(x \oplus \alpha), \qquad 0 \le x < 65536.$$

The preprint gives no search code, tested key values, or explicit ranges for the input words and differences. It is therefore unclear whether the reported maxima cover every input difference for one key, several keys, or any other set of cases.

If all input words are counted, 22 corresponds to

$$\frac{22}{65536} \approx 0.00033569 \approx 2^{-11.54}$$

for one 16-bit component. The seven pairs in Table III have no accompanying counts, leaving their relation to this maximum unclear.

The entry for zero layers gives another check. If it means removing the `b16` boxes from Figure 2 while retaining the final substitution and key XORs, the stated S-box maximum of four gives

$$4 \cdot 16^3 = 16384.$$

This differs from the reported value of 16370.

There is further ambiguity in the implementation. Section II-B lists four S-boxes but says that the implementation uses one S-box in place of four different ones [2]. The reference code examined here uses the four distinct tables. We use that reference implementation throughout this paper.

Section 3.2 derives a lower bound on the maximum difference count for this rotor by counting its possible output differences. This gives a direct comparison with the reported maximum.

1.5 Gaps in the current analysis

The missing analysis of the linear layer even where it applies to the differential resistance measured by the authors is what initially motivated this work. The following section will show that a thorough linear analysis of SEPAR reveals critical weaknesses.

2 SEPAR's design and the attack model

SEPAR encrypts 16-bit plaintext words sequentially using a 256-bit master key, with a state that includes a Linear Feedback Shift Register. The algorithm is designed for constrained devices, such as Internet of Things (IoT) devices. The authors report throughput improvements of up to 42.22% over BORON and 98.01% over PRESENT in some contexts [1]. We examine its security under chosen IV and chosen plaintext queries.

2.1 Definitions

For each 16-bit (*or word*) block, there are $2^{16} = 65536$ possible values. A *rotor* as it is called by the authors, is the keyed permutation applied 8 times to a word as it passes through SEPAR's encryption *cascade*.

Each rotor maps the 2^{16} inputs bijectively onto the 2^{16} outputs. Its key determines the mapping, and each of the eight rotors uses a disjoint 32-bit segment of the master key. The permutation has an *inverse*, which recovers the input from the output given the rotor key. Decryption uses this inverse.

Each rotor is itself built from a sequence of substitution, transposition, and bitwise operations, described in later sections, and between each rotor in the cascade is a state addition.

A *codebook* will refer to a table of queried plaintext words and their corresponding ciphertext words under one IV. A full codebook contains all 65536 input words, and a partial codebook contains only the queried subset.

Resetting the cipher clears the current state and computes new state words from the 128-bit *initialisation vector* (IV) through rotor operations keyed by the master key. This computation of the state before the first word will be referred to as *initialisation*.

This paper will examine the cipher under the Chosen IV, Chosen Plaintext attack model. Under this model, the attacker can choose an IV and a plaintext and obtain the first ciphertext word, with a reset before each query. This will be referred to as 'querying the oracle'. Under this model, our *oracle* is an interface that answers chosen queries without revealing any internal information - it accepts plaintext and an IV, and responds with only ciphertext. This is akin to a real world scenario where an attacker can query a device or service using the SEPAR protocol, assuming the implementation allows them to supply their own IV and plaintext.[4]

All word additions and subtractions are modulo 2^{16} *unless stated otherwise.*

2.2 The rotor permutation

First, write a word as four nibbles (a, b, c, d), from most to least significant. A nibble contains four bits, for example, `0x1234` has nibbles $1, 2, 3, 4$.

The high byte is (a, b), the middle byte is (b, c), and the low byte is (c, d) (*Note that these overlap*).

SEPAR's 4 S-boxes are substitution tables that work on the sixteen nibble values. Let Σ apply

the four S-boxes in parallel. Their entries are:

P	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
S_1	1	F	B	2	0	3	5	8	6	9	C	7	D	A	E	4
S_2	6	A	F	4	E	D	9	2	1	7	C	B	0	3	5	8
S_3	C	2	6	1	0	3	5	8	7	9	B	E	A	D	F	4
S_4	D	B	2	7	0	3	5	8	6	C	F	1	A	4	9	E

Rotor i (of which there are 8 applied to the plaintext block per encryption) has a 32-bit key which is divided into two 16-bit keys $K_i = (k_{i,0}, k_{i,1})$. The *key schedule* derives the values used in its internal layers from these two words. Write $\mathrm{rol}_{16}(u, r)$ for a 16-bit left rotation.

The four rotor subkeys are defined as:

$$\begin{aligned} G(u) &= u \text{ OR } (S_1((u \gg 6) \,\&\, 15) \ll 6), \\ \kappa_0 &= k_{i,0}, \qquad \kappa_1 = k_{i,1}, \\ \kappa_2 &= G(\mathrm{rol}_{16}(k_{i,0}, 6)) \oplus (i+2), \\ \kappa_3 &= G(\mathrm{rol}_{16}(k_{i,1}, 10)) \oplus (i+3). \end{aligned}$$

The linear layer L is the starting point of the attack.

Here "linear" means that applying the layer to an XOR of two inputs gives the XOR of their separate outputs: $L(x \oplus y) = L(x) \oplus L(y)$.

The linear layer mixes input bits so that later S-boxes combine information from different parts of the word.

Let (a, b, c, d) be the four nibbles entering the linear layer, after substitution.

The linear layer, which applies to the 16-bit plaintext word, made of four nibbles $P = a, b, c, d$ defines:

$$\begin{aligned} A &= a \oplus c, \\ B &= b \oplus d, \\ C &= c \oplus B = c \oplus b \oplus d, \\ D &= d \oplus A = d \oplus a \oplus c \end{aligned}$$

then, after $w = A \| B \| C \| D$, L is defined as;

$$L(a, b, c, d) = w \oplus \mathrm{rol}_{16}(w, 8) \oplus \mathrm{rol}_{16}(w, 12).$$

Expanding the XORs and rotations in this formula reveals cancellations that undo much of the mixing.

Writing their formula for L out in terms of nibbles;

$$\mathrm{rol}_{16}(w, 8) = (C, D, A, B), \qquad \mathrm{rol}_{16}(w, 12) = (D, A, B, C).$$

and therefore;

$$L(a, b, c, d) = (A \oplus C \oplus D,\ B \oplus D \oplus A,\ C \oplus A \oplus B,\ D \oplus B \oplus C).$$

Substituting back in the defined values for A, B, C and D;

$$\begin{aligned} A \oplus C \oplus D &= (a \oplus c) \oplus (c \oplus b \oplus d) \oplus (d \oplus a \oplus c) \\ &= b \oplus c, \\ B \oplus D \oplus A &= (b \oplus d) \oplus (d \oplus a \oplus c) \oplus (a \oplus c) \\ &= b, \\ C \oplus A \oplus B &= (c \oplus b \oplus d) \oplus (a \oplus c) \oplus (b \oplus d) \\ &= a, \\ D \oplus B \oplus C &= (d \oplus a \oplus c) \oplus (b \oplus d) \oplus (c \oplus b \oplus d) \\ &= a \oplus d. \end{aligned}$$

and thus L can be simplified as;

$$\boxed{L(a, b, c, d) = (b \oplus c,\ b,\ a,\ a \oplus d).} \qquad \square \tag{1}$$

The second output is simply b, and input d affects only the fourth output. We will follow these limited dependencies through the complete rotor.

Finally, Vahi and Jassbi assemble the components into the keyed rotor permutation, hereafter written $\texttt{Enc}_i$.

Starting with x, perform

$$x \leftarrow L\Sigma(x \oplus \kappa_j) \quad \text{for } j = 0, 1, 2, 3, \qquad x \leftarrow \Sigma(x \oplus \kappa_0 \oplus \kappa_1) \oplus \kappa_2 \oplus \kappa_3.$$

$\texttt{Enc}_i$ has an inverse $\texttt{Dec}_i$, defined in the cipher's proposal paper [1]. It reverses the order of operations and inverts each one, so $\texttt{Dec}_i(\texttt{Enc}_i(x)) = x$.

2.3 Initialisation & State

The *state* consists of eight 16-bit words. There is also a *Linear Feedback Shift Register* (LFSR), a state register updated by shifting bits and XORing selected feedback bits from the ciphertext. The state is initialised through rotor operations on the IV for the first block of encryption, and the LFSR is only used after this first encryption.

For an initialised state $s = (s_1, \ldots, s_8)$, the map is

$$R_0(p) = p, \qquad R_i(p) = \texttt{Enc}_i(R_{i-1}(p) + s_i), \qquad C(p) = R_8(p). \tag{2}$$

Here $R_i(p)$ is the intermediate word after rotor i, and $C(p)$ is the ciphertext word returned to the attacker. Each stage first adds its state word and then applies its keyed rotor. The first rotor is nearest the plaintext; the eighth is nearest the ciphertext. We call the latter the *outer rotor*.

The cascade can be read from left to right below. Each labelled arrow adds a state word before the next rotor.

$$p \xrightarrow{+\,s_1} \boxed{\texttt{Enc}_1} \xrightarrow{+\,s_2} \boxed{\texttt{Enc}_2} \longrightarrow \cdots \xrightarrow{+\,s_8} \boxed{\texttt{Enc}_8} \longrightarrow C(p).$$

For a fixed IV, all state words in (2) are constants. The observed map is therefore a composition of fixed additions and fixed permutations when using the same chosen IV.

Initialisation consists of four rounds of the map given later in Section 3.

3 Preserved classes and useful states

3.1 Weak rotor diffusion due to the Linear Layer

Diffusion spreads input information across the output, as described by Shannon in his seminal work *Communication Theory of Secrecy Systems* [5]. Equation (1) shows that one SEPAR layer mixes the nibbles poorly. We now follow those dependencies through the complete rotor to see whether its repeated substitutions and linear layer applications overcome this limitation.

Consider the 65536 possible input words. We can define a *partition* that divides them into disjoint groups, called *classes*. We say a prefix is a specified number of leading bits. For example, fixing a 12-bit prefix leaves only the low nibble free, so each prefix class contains $2^{16-12} = 16$ words. There are 4096 such classes. We use this partition and two others:

$$\begin{aligned} B_u &= \{16u + d : 0 \le d < 16\}, && 0 \le u < 4096, \\ H_v &= \{256v + r : 0 \le r < 256\}, && 0 \le v < 256, \\ M_w &= \{4096a + 16w + d : 0 \le a, d < 16\}, && 0 \le w < 256. \end{aligned}$$

Their indices are the 12-bit prefix $u = x \gg 4$, the high byte $v = x \gg 8$, and the middle byte $w = (x \gg 4)\ \&\ 255$, respectively. Each index is a specified part of the word x. The letters B, H, M will denote three different ways of defining a partition.

The 12-bit prefix fixes nibbles a, b, c and lets d vary. The high byte view fixes a, b, while the middle byte view fixes b, c.

View	**Word** (a, b, c, d)	**Class label and size**	**Use**
12-bit prefix B_u	a b c $*$	$u = 256a + 16b + c$ 4096 classes, 16 words each	Build cycles; verify candidates Sections 4–6
Middle byte M_w	$*$ b c $*$	$w = 16b + c$ 256 classes, 256 words each	Search rotor keys Sections 4–6
High byte H_v	a b $*$ $*$	$v = 16a + b$ 256 classes, 256 words each	Recover rotor two Section 7

Figure 1: The three views of a word. Shaded nibbles are fixed within a class; each $*$ can take all sixteen values. From a prefix label u, the middle byte is $w = u \bmod 256$ and the high byte is $v = \lfloor u/16 \rfloor$.

Astute readers may recognise our choice of nibbles here from the weakness in the Linear Layer. We now follow the input dependencies through the rotor to see that these three partitions are preserved.

A prefix class is *aligned* when its first value is a multiple of its size, as in B_u or H_v.

A class's *image* is the set of outputs obtained by applying the rotor to all its members. A rotor *preserves a partition* when it sends every whole input class to a whole output class. It can rearrange both the classes and the words inside them. The resulting rearrangement of class indices is the *induced permutation*. This describes the grouping without referring to how the individual words are ordered inside it. The following lemma proves that all three partitions are preserved.

Let P, Q, M, and T denote the 4 nibble output maps induced by a keyed rotor;

Lemma 3.1 (Rotor dependencies)**.** *For every rotor key,*

$$\mathtt{Enc}_i(a,b,c,d) = \big(P_{i,b}(a),\, Q_i(b),\, M_{i,b}(c),\, T_{i,a,b,c}(d)\big),$$

where each coordinate map is a permutation. The subscripts identify the fixed input nibbles; the argument is the nibble being varied. For example, $P_{i,b}$ permutes a for a fixed rotor i and second input nibble b. Consequently, the twelve most significant output bits depend only on the twelve most significant input bits. The induced permutation on middle bytes

$$m_i(16b + c) = 16Q_i(b) + M_{i,b}(c)$$

is a permutation of 256 values.

Proof. One keyed substitution and linear layer send (a, b, c, d) to coordinates depending respectively on $(b, c), b, a, (a, d)$. Four layers (the amount of applications specified per rotor) give dependency sets $(a, b), b, (b, c), (a, b, c, d)$. The final substitution and XOR do not enlarge them. With the other coordinates fixed, each coordinate calculation is a composition of nibble permutations and XORs. □

Substitution and XOR with a fixed subkey act separately on each nibble. Only the linear layer changes which input coordinates can influence an output. Applying (1) four times gives the dependencies in Table 3.

Layers completed	First output	Second output	Third output	Fourth output
0	a	b	c	d
1	b, c	b	a	a, d
2	a, b	b	b, c	a, b, c, d
3	b, c	b	a, b	a, b, c, d
4	a, b	b	b, c	a, b, c, d

Table 3: *Input nibbles that can influence each output after successive substitution and linear layers. The final substitution and key XORs do not introduce further dependencies.*

In particular, changing d cannot change the first three output nibbles, and changing c, d cannot change the first two. The middle output byte depends only on b, c, so both a and d can be discarded when studying that byte. Figure 2 shows these restrictions for the complete rotor.

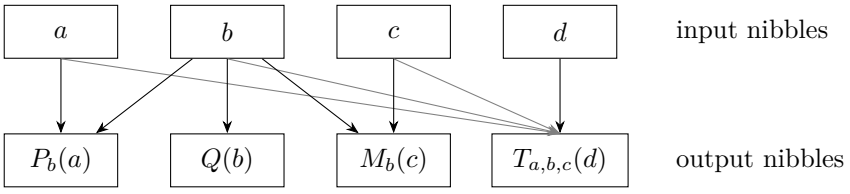


Figure 2: *Dependencies in a complete rotor. The low input nibble d has no path to any of the first three outputs.*

Unlike prefix classes, the middle byte classes are not consecutive intervals of word values: their top and bottom nibbles both vary. We use the three views as follows:

View	Retained nibbles	Indices	Main use
12-bit prefix	a, b, c	4096	Cycles of class indices
Middle byte	b, c	256	Recovering rotor keys
High byte	a, b	256	Final two rotors

The dependencies ensure that all words in one input class have the same output index. Since the rotor is invertible and the classes have equal size, different input classes must go to different output classes. We can therefore treat each class as one object in a smaller permutation.

3.2 Consequences for differential analysis

The rotor dependencies let us test the claims discussed in Section 1. We first check the local S-box probability, then examine how differences pass through the complete rotor.

Direct enumeration of the four S-boxes in Section 2 gives

$$\max_{\alpha \neq 0,\, \beta} D_{S_j}(\alpha, \beta) = 4, \qquad j = 1, 2, 3, 4.$$

Their maximum transition probability is therefore $4/16 = 2^{-2}$.

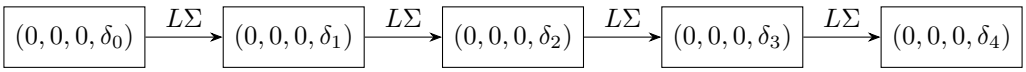
The rotor defined in Section 2 gives a direct way to test the active S-box claim. Take any pair of inputs whose XOR difference is confined to the low nibble:

$$\Delta = (0, 0, 0, \delta), \qquad \delta \neq 0.$$

The first three S-box inputs agree. Only S_4 is active, and its output difference δ' is nonzero because S_4 is a permutation. Equation (1) then gives

$$L(0, 0, 0, \delta') = (0, 0, 0, \delta').$$

The same argument applies at every internal layer, regardless of the subkeys. The difference never leaves the low nibble. Its value can depend on the inputs and subkeys, but its position and the fact that it is nonzero do not. Figure 3 shows this.



Only S_4 is active in each layer; every $\delta_j \neq 0$.

Figure 3: *A difference confined to the low nibble through four internal rotor layers. Subkey XORs are omitted because they cancel in the difference. This gives four active S-boxes in total.*

Thus four consecutive substitution and linear layers have a valid trail with exactly four active S-boxes. The final substitution adds one more, giving five across the complete rotor. This contradicts a universal lower bound of ten for the four internal layers of the rotor analysed here.

The preserved prefixes also allow us to verify the differential tables for the rotor. Fix a key and a nonzero difference α confined to the low nibble. For every input x, Lemma 3.1 ensures that the outputs for x and $x \oplus \alpha$ have the same first twelve bits. Since the rotor is bijective, their output difference is nonzero. There are therefore only fifteen possible output differences, all in the low nibble.

The 65536 ordered inputs must be distributed among those fifteen possibilities, so at least one output difference satisfies by pidgeonhole;

$$\max_{\beta \neq 0} D_{\mathtt{Enc}_i}(\alpha, \beta) \geq \left\lceil \frac{65536}{15} \right\rceil = 4370.$$

If the reported value 22 is intended as the maximum unnormalised difference count for this same rotor, it is incompatible with that lower bound.

This counterexample concerns one rotor. It remains to be shown whether these findings permit a practical differential attack on the cipher.

3.3 How IVs determine state

The preserved classes encode information about the rotors' keying (since it is the subkey that determines their relations), but the rotors are not the only component of the cipher. Whether the classes are easy to observe through the whole cascade depends on the state additions as well.

Initialisation starts with the eight words of the IV and applies four rounds using the same keyed rotors as the encryption cascade. One initialisation round first computes

$$v_1 = \mathtt{Enc}_1(s_1 + s_3 + s_5 + s_7), \qquad v_i = \mathtt{Enc}_i(v_{i-1} + s_i) \quad (2 \le i \le 8),$$

then replaces s_1 by $s_1 + v_8$ and s_i by $s_i + v_{i-1}$ for $i \ge 2$.

Lemma 3.2 (Uniform state)**.** *For every fixed key, four initialisation rounds permute the 128-bit IV space. Independent uniform IVs therefore give independent uniform initialised state words.*

Proof. For $i \ge 2$, the new state is $t_i = s_i + v_{i-1}$, which is also the input used to compute v_i. Hence $v_i = \mathtt{Enc}_i(t_i)$. Given the output t of one round, invert in the order

$$s_1 = t_1 - \mathtt{Enc}_8(t_8), \quad s_i = t_i - \mathtt{Enc}_{i-1}(t_{i-1}) \ (i = 8, \dots, 3), \quad s_2 = t_2 - \mathtt{Enc}_1(s_1 + s_3 + s_5 + s_7).$$

Four bijections compose to a bijection. □

Thus, for a fixed key, every possible state comes from exactly one IV. Uniform IV sampling therefore lets us count the probability of the state patterns used in the attack.

3.4 Carries reveal adjacent classes

The state addition before a rotor can move word values across class boundaries. For example, adding six to $B_2 = \{32, \dots, 47\}$ produces the integers 38 through 53: ten belong to B_2 and six to B_3. We call these classes *adjacent* because their indices are consecutive. At the end of the 16-bit range, addition wraps to zero, so the last class and class zero are adjacent too. Figure 4 shows the split that occurs when an offset is added to a class.

Suppose the next rotor sends B_2 to B_{41} and B_3 to B_{46}. The split then appears as ten output values in class 41 and six in class 46. These labels depend on the rotor key. The difficulty is recognising the pair after earlier rotors and additions have disturbed some of the words. We address this by counting how many words can leave their class.

An input *row* is a set of consecutive plaintexts that we examine together. For rows of length B, its correct alignment makes the first addition land on a whole class of B words. A *translation* means adding the same value to every input (with wrap around). A translation by a multiple of B then moves a class without splitting it. Any remaining part can move some word values into an adjacent class. We also call word values *points* when discussing sets. The rotor that follows only renames the classes and rearranges their contents.

We can then measure a small remainder in either direction. For instance, $159 = 16 \cdot 10 - 1$: it is ten whole classes followed by a displacement of minus one. Write an intermediate state word as $Bh_i + e_i$, where e_i is such a remainder. The number $S = \sum_i |e_i|$ bounds how many row points can have left their intended class before the final addition.

We call a point *disturbed* when an intermediate addition's remainder moves it outside the intended class.

For a row, count how many of its outputs fall in each class. We rank classes by these counts.

Lemma 3.3 (Two largest counts)**.** *Let B be 16 or 256, so every rotor preserves aligned classes of B words. Align a row of B plaintexts with the first addition. Before the last rotor, suppose the intermediate additions have remainders whose absolute values sum to S. Let b, with $0 < b < B$, be the remainder of the last addition. If*

$$2S < \min(b, B - b), \tag{3}$$

the two output classes with largest row counts are exactly the images of two adjacent input classes of the last rotor. Together they contain at least $B - S$ row points. After inverting that rotor, at least $B - S$ points lie in one interval $[Bv + b, Bv + b + B - 1]$, modulo the word size.

Proof. After the first rotor, the aligned row fills one class. Adding $Bh_i + e_i$ can move at most $|e_i|$ of that class's points out of its intended translated class. Discard those points. The next rotor preserves the intended class, and cannot increase the number discarded. Repeating the argument leaves at least $B - S$ points in the intended class before the last addition.

A complete class would be split by the final addition into parts of sizes $B-b$ and b. Each intended output class therefore contains at least its complete class size minus S. All other classes together contain at most S points. The strict inequality (3) makes both intended counts greater than S, so these are precisely the two largest classes. The retained points are also in the translated interval before the final rotor. □

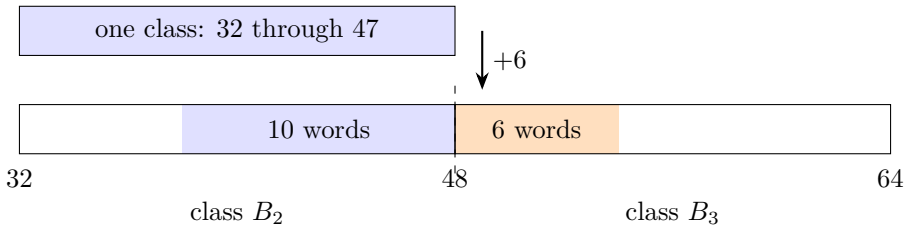


Figure 4: Adding six splits a class of sixteen words between two adjacent classes. If earlier additions have disturbed at most two words, the intended counts remain at least eight and four, while all other classes together contain at most two. The two largest counts still identify the adjacent pair.

For $B = 16$, condition (3) implies $S \leq 3$ as the maximum value of $\min(b, 16 - b) = 8$. The count is recorded in an undirected graph where each vertex is an observed output class index, and an edge joins the two indices occurring most often in one row.

When every adjacent input pair has been observed, a hidden induced permutation q on n class indices gives the cycle;

$$q(0), q(1), \ldots, q(n-1), q(0).$$

For example, the order $2, 0, 3, 1, 2$ gives edges $\{2, 0\}, \{0, 3\}, \{3, 1\}, \{1, 2\}$. Following the edges recovers the order, but leaves an unknown *origin*, the vertex corresponding to input zero, and an unknown *orientation*, which direction corresponds to increasing input values. The attack tries both directions and all compatible origins.

4 Generating rotor key candidates

Section 3 shows how adjacent classes reveal a rotor's cyclic order. We now turn that order into key candidates. The starting input and direction are unknown, and an inner rotor may also be followed by an unknown addition. The second nibble gives us a way into the problem: its sixteen

outputs depend on just four key nibbles. We search these first, use the key schedule to extend them to complete rotor keys, and compare the resulting permutations with the observed cycle. *Throughout this section, byte values and positions are reduced modulo 256, and prefix values and positions modulo 4096.*

4.1 The second nibble

Write the four round subkeys as $\kappa_j = (\alpha_j, \beta_j, \gamma_j, \delta_j)$, for $j = 0, 1, 2, 3$. Only the four β_j's affect the second output nibble. Each round XORs this nibble with β_j, applies S_2, and leaves the result unchanged in the linear layer. For input b, its path is

$$\begin{aligned} b_0 &= S_2(b \oplus \beta_0), \\ b_1 &= S_2(b_0 \oplus \beta_1), \\ b_2 &= S_2(b_1 \oplus \beta_2), \\ b_3 &= S_2(b_2 \oplus \beta_3), \\ Q(b) &= S_2(b_3 \oplus \beta_0 \oplus \beta_1) \oplus \beta_2 \oplus \beta_3. \end{aligned}$$

There are $16^4 = 65536$ choices of these four nibbles. For each choice, we calculate the sixteen values of Q.

4.2 Reading the middle byte cycle

An input middle byte is $16b + c$. Fixing b and letting c run from zero to fifteen gives a *block* of sixteen consecutive input bytes. Its outputs all have the same second nibble, $Q(b)$, while the third output nibble takes every value. The cycle therefore consists of sixteen blocks of length 16, each with a different constant second output nibble. Adding a multiple of sixteen to the output byte changes those labels but preserves the blocks.

We can locate a block boundary by finding where the second output nibble changes. Start the observed list immediately after such a change, and check that it has sixteen blocks of length 16 with distinct labels. Call these labels $O(0), \ldots, O(15)$.

We still need to find the starting input block j, the direction ε, and the addition λ to the second output nibble. Try each $j \in \{0, \ldots, 15\}$ and $\varepsilon \in \{1, -1\}$. For a proposed Q, the first label fixes

$$\lambda = O(0) - Q(j) \pmod{16}.$$

Check the other labels using

$$O(k) = Q(j + \varepsilon k) + \lambda \pmod{16}, \qquad 0 \le k < 16. \tag{4}$$

The input to Q is also reduced modulo 16. At the outer rotor there is no following addition, so we require $\lambda = 0$.

For example, a forward cycle starting at input block nine, with an output addition of three, has labels $Q(9) + 3, Q(10) + 3, \ldots, Q(8) + 3$, reduced modulo sixteen. The guess $j = 9, \varepsilon = 1$ determines the addition from the first label, and the remaining labels test that guess.

4.3 Extending guesses to rotor keys

The four guessed β nibbles restrict the original key words (k_0, k_1). The schedule derives κ_2 from k_0 and κ_3 from k_1, so we can search the words separately:

1. Try all 65536 values of k_0. Keep those whose second nibble is β_0 and whose scheduled κ_2 has second nibble β_2.
2. Do the same for k_1, using β_1 and β_3.
3. Pair the surviving words and compare each resulting rotor with the observed middle byte cycle.

Enumerating the possible words gives at most 576 entries in each list for this schedule. Each surviving nibble guess therefore leaves at most $576^2 = 331776$ key pairs to test.

Write $Y_0, \ldots, Y_{255}$ for the observed middle bytes, starting at the block boundary chosen above. The first input byte is

$$v = \begin{cases} 16j, & \varepsilon = 1, \\ 16j + 15, & \varepsilon = -1. \end{cases}$$

In the forward direction we enter a block at $c = 0$; in reverse we enter at $c = 15$. Position x then corresponds to input $v + \varepsilon x$, modulo 256.

For a candidate key $K = (k_0, k_1)$, calculate

$$m_K(z) = (\mathtt{Enc}_K(16z) \gg 4) \,\&\, 255.$$

When $z = 16b + c$, the word $16z$ has nibbles $(0, b, c, 0)$. By Lemma 3.1, the top and bottom input nibbles do not affect this middle output byte. The complete comparison is

$$m_K(v + \varepsilon x) + 16\lambda \equiv Y_x \pmod{256}, \qquad 0 \le x < 256. \tag{5}$$

Keep every key pair that agrees at all 256 positions.

Theorem 4.1 (Completeness of the key search)**.** *Given a middle byte rotor cycle up to an input origin, direction, and an output multiple of 16, the procedure above returns every compatible rotor key and that output shift.*

Proof. Take any compatible key. The search tries its four second subkey nibbles, its starting block, and its direction. Equation (4) then gives its output addition. Both original key words pass the schedule checks, so their pair reaches (5). It agrees with the full cycle and is retained. □

The lists of key words can be prepared once and reused. We also group the nibble guesses by their first three output differences,

$$(Q(j + \varepsilon) - Q(j),\ Q(j + 2\varepsilon) - Q(j),\ Q(j + 3\varepsilon) - Q(j)) \pmod{16}.$$

Subtracting the first output removes λ, so the observed differences select the guesses to compare against all sixteen labels.

4.4 Checking the full prefix cycle

When all 4096 prefix labels are available, the top nibble gives a further check on the candidates. Write $q_K(x) = \mathtt{Enc}_K(16x) \gg 4$. With starting input a, direction ε, and a following addition h, the observed prefix cycle has the form

$$q_K(a + \varepsilon x) + h \pmod{4096}, \qquad h = \ell + 16\lambda + 256H. \tag{6}$$

Each of ℓ, λ, H is a nibble. We recover them in three steps:

1. Try each ℓ, subtract it from the observed cycle, and keep the low byte. This must give sixteen copies of one 256-entry cycle, since the middle byte does not depend on the top input nibble.
2. Apply the middle byte search to one copy. It returns candidate keys and the middle nibble λ of the addition.
3. For each result, try all sixteen H's. Locate $q_K(0) + h$ in the full cycle and compare all 4096 entries in both directions.

The value at input zero fixes the origin for the last comparison. Thus the same search recovers candidates for a rotor alone or for a rotor followed by an addition.

We will now show how we can use these lemmas and algorithms to obtain a cycle, apply the keysearch, and then undo the outermost rotor to work our way through the cipher.

5 Recovering the outer rotor

We must treat rotor 8 (the outermost rotor) as a special case since there is no preceding state addition. We can apply the key search directly to a cycle built from ciphertext, and sample IVs until the state additions leave enough of each plaintext class together for Lemma 3.3 to apply.

5.1 When the cycle is visible

Group plaintexts into rows of sixteen. The correct alignment is $r = -s_1 \bmod 16$, so we try all sixteen values of r. For the six intermediate additions, write $s_i = 16h_i + e_i$, choosing the remainder nearest zero, and set

$$S = \sum_{i=2}^{7} |e_i|, \qquad b = s_8 \bmod 16.$$

Lemma 3.3 identifies the images of adjacent classes whenever

$$2S < \min(b, 16 - b). \tag{7}$$

Call this the *outer event*. If $S = 0$, the intermediate additions preserve whole classes and the last addition splits them. The lemma also allows up to three disturbed words, giving us more useful IVs.

For each of 256 aligned rows, select the two most frequent ciphertext prefixes and reduce their labels modulo 256 to obtain middle bytes. Join each pair by an edge. On the outer event, the rows cover all adjacent pairs of middle byte classes, giving the cycle needed by Section 4. We check that every vertex has two neighbours and that following the edges visits all 256 vertices before returning to the start.

5.2 Applying the key search

Apply Theorem 4.1 with no output addition. For each candidate, invert the rotor on the complete words in the rows. The two dominant prefix classes must become adjacent. The words must also fit the split caused by the last state addition.

To test this, try $b = 1, \ldots, 15$ and let

$$L(b) = \min\left(3, \left\lfloor \frac{\min(b, 16 - b) - 1}{2} \right\rfloor\right). \tag{8}$$

This is the largest number of disturbed words allowed by (7). Require at least $16 - L(b)$ words in an interval $[16v + b, 16v + b + 15]$, modulo 65536, in every row. The class index v may vary between rows but the same b must work throughout. Retain every key and b that pass.

Theorem 5.1 (Outer rotor recovery)**.** *On event* (7)*, the cycle search and row test retain the true K_8. For every fixed master key, a uniform IV satisfies this event with probability*

$$p_{\text{out}} = \frac{1527}{16^7} \approx 5.68852 \times 10^{-6}. \tag{9}$$

Proof. The graph gives the true middle byte cycle, so Theorem 4.1 retains the true key. After its inverse, Lemma 3.3 gives adjacent classes and at least $16 - S$ words in the required interval. Since $S \le L(b)$, the row test passes.

To count the states, group the six remainders by their total absolute value S:

S	nonzero magnitudes	choices	allowed b
0	none	1	15
1	1	$6 \cdot 2 = 12$	11
2	2 or $1 + 1$	$6 \cdot 2 + 15 \cdot 4 = 72$	7
3	3, $2 + 1$, or $1 + 1 + 1$	$6 \cdot 2 + 30 \cdot 4 + 20 \cdot 8 = 292$	3

For example, $2 + 1$ has six choices for the position of the two, five for the one, and four choices of signs. For each S, the condition $2S < b < 16 - 2S$ allows $15 - 4S$ values of b. By Lemma 3.2, the seven low nibbles are uniform, giving

$$p_{\text{out}} = \frac{1 \cdot 15 + 12 \cdot 11 + 72 \cdot 7 + 292 \cdot 3}{16^7} = \frac{1527}{16^7}.$$

□

5.3 Screening IVs

Most IVs can be rejected after a few queries. Before collecting the cycle, query strips of 31 consecutive plaintexts at twelve fixed positions. Each strip contains a complete row at every possible alignment. If $A \ge B$ are the two largest prefix counts, require

$$A + B \ge 13, \qquad B > 16 - A - B.$$

These tests allow at most three words outside the dominant pair and require the smaller count to exceed the total outside it. We first check subsets of six and nine words, requiring the two largest counts to sum to at least the subset size minus three.

For an alignment that survives all twelve strips, query $p = 0, \ldots, 4110$. These $4111 = 256 \cdot 16 + 15$ words supply the 256 rows at every alignment. Run the cycle search and row test on each surviving alignment.

6 Recovering the remaining rotors

Once we have a candidate for K_8, we can invert the outer rotor:

$$\mathtt{Dec}_8(C(p)) = R_7(p) + s_8.$$

The next rotor is now visible through an unknown addition. Section 4 already handles this situation. We recover candidates for K_7 and s_8, subtract the addition, and invert rotor seven. Repeating this idea takes us inward to K_3. The first two rotors need a change of class size, described in Section 7.

6.1 Keeping track of the state

If the current table is $Y_t(p) = R_t(p) + s_{t+1}$, one step gives

$$\mathtt{Dec}_t(Y_t(p) - s_{t+1}) = R_{t-1}(p) + s_t. \tag{10}$$

The rotor keys are shared by all IVs, while the state additions belong to one IV. We therefore save each useful IV and its codebook together with the keys and additions used to transform it. We call this a *record*, and retain every surviving one to ensure the true branch (if it is included) is not dropped.

Write a state word as $s_i = 16h_i + \ell_i$, with $0 \le \ell_i < 16$. After trying and subtracting the bottom nibble ℓ_i, the remaining addition shifts each prefix label by h_i. This is the output shift in Section 4.4.

6.2 States that expose an inner rotor

To reach an inner rotor, look for a state in which one addition makes a small split and the surrounding additions preserve classes. For a target $t \in \{6, 5, 4, 3\}$, define the event E_t by

$$\begin{gathered} s_i \bmod 16 = 0 \quad (2 \le i \le 6,\ i \ne t), \qquad e_t \in \{-3, -2, -1, 1, 2, 3\}, \\ 2|e_t| < \min(b_7, 16 - b_7), \end{gathered} \tag{11}$$

where e_t is the remainder of s_t, and $b_7 = s_7 \bmod 16$. For example, remainders $(s_2, s_3, s_4, s_5, s_6, s_7) = (0, 0, 0, 2, 0, 6)$ satisfy E_5.

This state serves two purposes. First, at most $|e_t|$ words leave their intended class before rotor seven. The inequality lets us apply Lemma 3.3 there, just as at the outer rotor. After inverting rotor eight, try the bottom nibble of s_8, build the cycle, and use Section 4.4 to find K_7 and the rest of s_8. Subtracting s_8 and inverting rotor seven leaves a record of $R_6(p) + s_7$.

Second, the addition before rotor t splits a whole class into exactly $16 - |e_t|$ words and $|e_t|$ words. The later additions through s_6 are multiples of sixteen, so they and the intervening rotors only permute the two classes. Removing those later operations reveals rotor t's cycle, ready for the same key search.

For $|e_t| = 1, 2, 3$, there are eleven, seven, and three choices of b_7, respectively, and two signs for e_t. Six low nibbles are constrained, so Lemma 3.2 gives

$$q = \Pr(E_t) = \frac{2(11 + 7 + 3)}{16^6} = \frac{42}{16^6}. \tag{12}$$

This is about one useful IV in 399458 for each target rotor.

We sample uniform IVs and screen them as in Section 5, trying each bottom nibble of s_8. A surviving IV is tested on the 4111-word partial codebook. After a candidate K_7, s_8 is removed, try the bottom nibble of s_7. Every row must then contain exactly two prefix classes, with one consistent split: 15 and 1, 14 and 2, or 13 and 3. Complete the codebook for records that pass, and check the same split on every row.

6.3 Finding the state additions

An E_6 record now gives K_6 directly by Section 4.4. For E_5, E_4, E_3, we also need to undo the already recovered rotors between the target and rotor six. Their state additions are unknown for the new IV.

Subtract a retained bottom nibble of s_7 and build the prefix cycle. All remaining additions after the target rotor are now multiples of sixteen. Write each as $16h$. Only h mod 256 affects the middle byte, so we can find this part first and leave the top nibble for later.

For example, with one known rotor outside the target, the middle byte cycle has the form

$$W(x) = m_1(m_0(x) + h_0) + h_1 \pmod{256},$$

where m_0 is the unknown rotor's middle byte map and m_1 is known.

Try each byte value of h_1 and calculate

$$V(x) = m_1^{-1}(W(x) - h_1).$$

The correct guess leaves $m_0(x) + h_0$, which is the single rotor problem from Section 4.

With d known rotors, repeat this subtraction and inversion from outside inward, trying all 256^d combinations of byte additions. Here $d \leq 3$: the largest search is for K_3 through K_4, K_5, K_6, giving $256^3 = 2^{24}$ combinations. The middle byte repeats every 256 positions of the prefix cycle, so one repetition is enough for this search.

A short check rejects most guesses before the key search. Within an input block, the sixteen outputs of $m_0 + h_0$ occupy sixteen consecutive byte values, possibly wrapping through zero. Define circular distance by

$$D(a, b) = \min\big((a - b) \bmod 256,\ (b - a) \bmod 256\big).$$

Of any sixteen consecutive steps in the cycle, only one crosses an input block boundary. At most one step can therefore have distance greater than fifteen. Reject guesses with two such steps in a group.

For each surviving list V, write $h_0 = f + 16\lambda$ modulo 256. Try the sixteen values of f, subtract each from V, and apply Theorem 4.1. This gives the target rotor key and λ, completing the byte parts of the additions.

It remains to recover their top nibbles. The middle byte comparison fixes a cycle position a whose input middle byte is zero, and a direction ε. Try the sixteen possible top input nibbles u and the top nibbles of the first d additions. The last addition is determined by one observed output. There are at most $16^{d+1} \leq 65536$ combinations.

More precisely, let F be the proposed chain of prefix permutations with the last addition set to zero, and now let W denote the full prefix cycle. Set

$$h_d = W(a) - F(256u) \pmod{4096},$$

check that its low byte agrees with the earlier result, and require

$$F(x + 256u) + h_d = W(a + \varepsilon x) \pmod{4096} \qquad (0 \leq x < 4096).$$

Finally, apply the proposed inverses and additions to the complete words and repeat the row test from Section 5. Save every key and set of additions that pass.

Lemma 6.1 (Correct additions survive the search)**.** *When E_t holds and the previously recovered keys and additions are correct, the search retains the true K_t and the intervening state additions.*

Proof. The byte search includes the actual additions. Undoing them leaves the target rotor followed by h_0. Its outputs within each input block lie in an interval of sixteen values, so it passes the distance check. Trying the actual f lets Theorem 4.1 recover the true key and λ.

The remaining guesses include the actual top nibbles. These determine the last addition and reproduce every prefix entry. After the correct operations are undone on complete words, each row is the translated interval produced by the single split at rotor t, so it passes the row test. □

We can now use an E_6 record to recover K_6, then records for E_5, E_4, E_3 to recover K_5, K_4, K_3 in order. Each step supplies the known rotor needed by the next.

7 Recovering the first two rotors

After finding K_3 on an E_3 record and undoing the later operations, we have

$$Z(p) = \mathtt{Enc}_2\big(\mathtt{Enc}_1(p + s_1) + s_2\big) + s_3.$$

We also know the input alignment $r = -s_1 \bmod 16$ and have candidates ℓ for the bottom nibble of s_3.

Rotor two. Here s_2 is a multiple of sixteen, so it preserves classes of sixteen words. To obtain a split, use rows of 256 words and require $s_2 \bmod 256 \neq 0$. A multiple of sixteen can still split one of these larger classes: adding sixteen, for example, gives 240 words in one high byte class and sixteen in the next.

Try each extension $r_{256} = r + 16j$ of the input alignment and $c = \ell + 16v$ of the output residue, with $0 \leq j, v < 16$. For each row starting at $r_{256} + 256u$, count the labels

$$((Z(p) - c) \bmod 65536) \gg 8.$$

With the correct guesses, rotor one gives a complete high byte class, and adding s_2 splits it between two neighbours. Their images under rotor two give the edges of a 256-entry cycle. The remaining output addition is $D = (s_3 - c)/256$.

We can use the same key guesses based on the second nibble as before. In the high byte view, the second nibble is the low nibble of each label. Its sequence repeats every sixteen entries, so use one repeat to search for the four key nibbles, allowing an unknown starting point, direction, and output addition. Extend the guesses to full rotor keys as in Section 4. Try all sixteen values for the upper nibble of D, and keep only matches on all 256 cycle entries. Each match gives K_2 and $s_3 = c + 256D$.

Rotor one. Undo each candidate to get

$$U(p) = \mathtt{Dec}_2(Z(p) - s_3) = \mathtt{Enc}_1(p + s_1) + s_2$$

for the correct guesses. Use sixteen word rows again, with starts $16u$ and $16u + 1$. After adding s_1, these alignments differ by one, so at least one splits each row between neighbouring classes.

Since s_2 is a multiple of sixteen, the prefix cycle search gives candidates for K_1 and s_2. The remaining addition follows from the entry at plaintext zero:

$$s_1 = \mathtt{Dec}_1(U(0) - s_2).$$

Check all 65536 entries of U. Then verify each complete master key against the saved codebooks, recomputing initialisation from each IV.

Lemma 7.1 (Recovery of the first two rotors)**.** *Given the correct keys and additions for the later rotors, E_3 together with $s_2 \bmod 256 \neq 0$ suffices to retain the true remaining keys and additions.*

Proof. The searches for rotor two include its true alignment, output addition, and key. They reproduce its cycle, so they are retained. After undoing them, at least one of the two row alignments reveals rotor one's cycle. Its true key and additions pass the key search and all checks on complete words. □

Under E_3, the bottom nibble of s_2 is zero, while the next nibble is still uniform. Fifteen of its sixteen values make the low byte nonzero. The event needed for these final steps therefore has probability

$$\frac{42}{16^6} \cdot \frac{15}{16} = \frac{630}{16^7}.$$

Rotor one requires no further state condition.

8 Recovery algorithm and success probability

The complete recovery has three stages:

1. Use the outer IV budget to collect candidates for K_8. For each candidate, use a separate inward IV budget to find K_7 and save the records from Section 6.
2. Extend each surviving pair one key at a time, finding K_6, K_5, K_4, K_3. Keep every matching key and its state additions.
3. Recover K_2, K_1 using Section 7. Check each complete key against every saved codebook, including initialisation from its IV.

The true key survives whenever the samples contain each of these state patterns:

Event	Information supplied	Probability per IV
Outer event	K_8 candidate	$1527/16^7$
E_6	K_7, then K_6	$42/16^6$
E_5	K_5	$42/16^6$
E_4	K_4	$42/16^6$
E_3, with $s_2 \bmod 256 \neq 0$	K_3, then K_2, K_1	$630/16^7$

Probability of retaining the true key

For every 256-bit master key, use m independent uniform outer IVs and N independent uniform inward IVs for each outer candidate. The probability that the final list omits the true master key is at most

$$\epsilon(m, N) = \left(1 - \frac{1527}{16^7}\right)^m + 3\left(1 - \frac{42}{16^6}\right)^N + \left(1 - \frac{630}{16^7}\right)^N. \tag{13}$$

At $m = 4205000$, $N = 7900000$, this is

$$\epsilon(m, N) \approx 1.6638512940036 \cdot 10^{-8} < 1.663852 \cdot 10^{-8}. \tag{14}$$

$$\boxed{\Pr\{\text{the algorithm returns the true key}\} \geq 1 - 1.663852 \cdot 10^{-8}.}$$

The output is a list of master keys that pass the checks and reproduce the saved codebooks from their IVs. All 500 measured runs returned a single key. A general bound on the number of surviving candidates remains open.

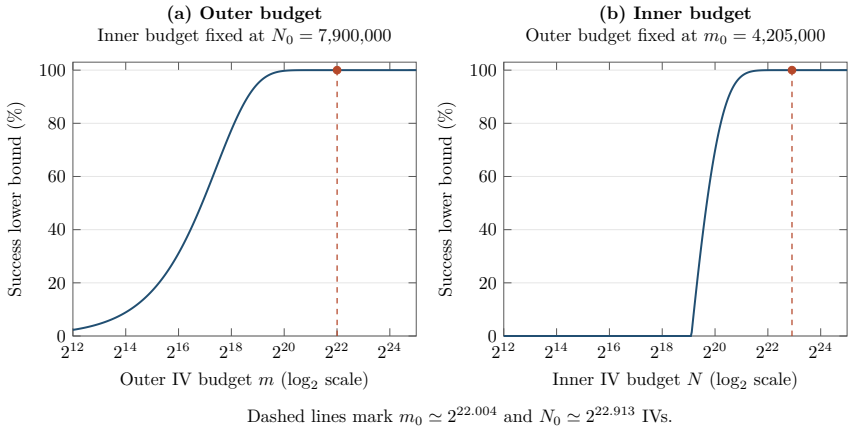


Figure 5: The lower bound on success probability as the IV budgets vary.

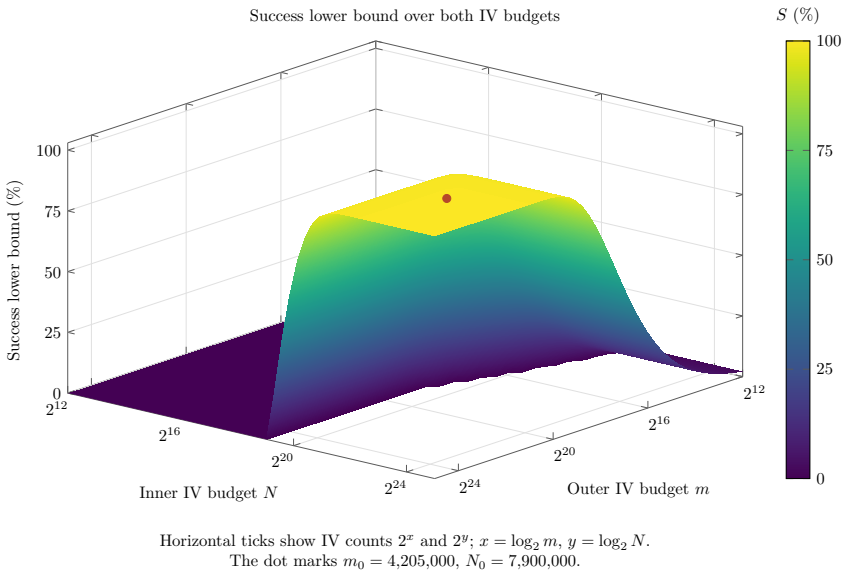


Figure 6: The success bound as a function of both IV budgets.

8.1 Query and computation costs

The implementation uses 4205000≈ (2^{22} outer IVs, and 7900000≈ (2^{23} inward IVs per each outer candidate (of which there has only been 1 observed per run).

Figures 5 and 6 show how the success bound changes with the two budgets.

For example, using one quarter of each budget gives a success lower bound of about 96.64%. The larger budgets reduce the failure bound to the value in (14).

Let B_o and B_i be the outer and inward budgets, Q be the number of total oracle queries, and let C be the number of outer candidates. Assuming the absolutely worst case bound where screening and codebook collection use at most 4483 queries per outer IV and 66032 per inward IV, giving

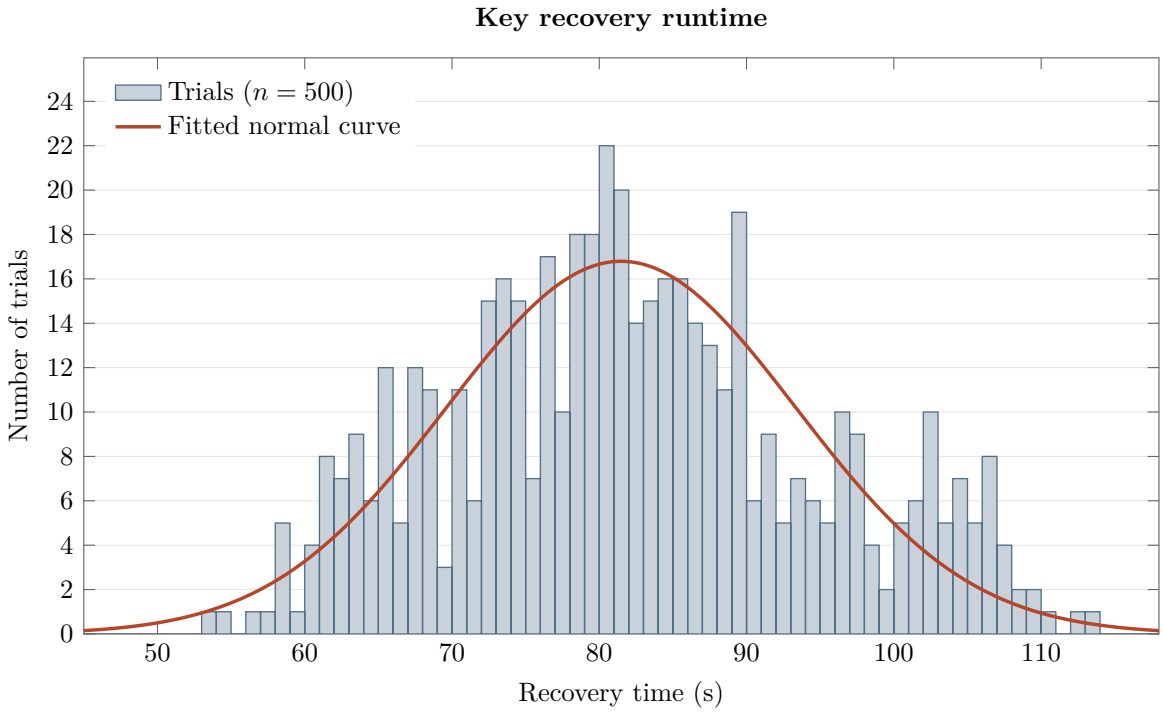
$$Q \leq 4483B_o + 66032B_iC.$$

At the stated budgets, this is

$$Q \leq 18851015000 + 521652800000C \approx 2^{34.13} + 2^{38.92}C.$$

This does not represent a typical run however. In 500 runs on randomly generated keys, every run produced only a single outer candidate, and most IVs were rejected after a few queries by the smart filtering described above.

The average was $2^{27.83}$ queries per run. The measured runtimes below use a single threaded of an AMD Ryzen 9 8945HS processor running at about 4 GHz.



1-second buckets. $\mu = 81.481$ s, $\sigma = 11.879$ s.

9 Conclusion

In conclusion, this work demonstrates a key recovery attack against SEPAR well below bruteforce given a chosen IV chosen plaintext oracle. We have demonstrated insufficient diffusion in the Linear Layer derived from the ciphers own algebra, and shown how that diffusion can be exploited at both the rotor level, and the whole Encryption block level to recover the full key with negligible chance of failure.

The resulting attack recovers candidates for the full master key under chosen IV and chosen Plaintext queries. With the stated budgets, the probability of omitting the true key is below 1.663852×10^{-8}, and the algorithm runs at the second scale on consumer hardware.

Work remains to be done to bound the number of surviving keys and establish the existence of a practical differential attack on the full cipher.

References

[1] A. Vahi and S. Jafarali Jassbi. SEPAR: A New Lightweight Hybrid Encryption Algorithm with a Novel Design Approach for IoT. *Wireless Personal Communications* 114, 2283–2314, 2020. doi:10.1007/s11277-020-07476-y.

[2] A. Vahi and M. Mirnia. On the Differential Cryptanalysis of SEPAR Cipher. arXiv:2106.12638, 2021.

[3] H. M. Heys. Key Dependency of Differentials: Experiments in the Differential Cryptanalysis of Block Ciphers Using Small S-boxes. *Cryptology ePrint Archive*, Report 2020/1349, 2020. eprint.iacr.org/2020/1349.

[4] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996. Chapter 7.

[5] C. E. Shannon. Communication Theory of Secrecy Systems. *Bell System Technical Journal*, 28(4), 656–715, 1949. `https://doi.org/10.1002/j.1538-7305.1949.tb00928.x`.